

## Ch02. 자연어와 단어의 분산 표현



## ■ 주요 내용

- 자연어 처리란
- 시소러스
- 통계 기반 기법
- 통계 기반 기법 개선하기

# 자연어와 단어의 분산 표현

- 자연어 처리가 다루는 분야
  - 컴퓨터가 사람의 말을 이해하게 만드는 것
  
- 이번 장에서 다룰 내용
  - 존재하는 자연어 처리 방법
  - 고전적인 자연어 처리 방법 (딥러닝 등장 이전의 기법)
  - 파이썬으로 텍스트 처리 구현

## 2.1 자연어 처리란

- Natural Language Processing (NLP)
  - 자연어를 처리하는 분야
  - 사람의 말을 컴퓨터에게 이해시키기 위한 기술
- 프로그래밍 언어
  - 기계적이고 고정 되어 있음
  - 딱딱한 언어
  - 문법이 정의되어 있고, 컴퓨터는 정해진 규칙에 따라 코드를 해석
- 자연어
  - 부드러운 언어 ( 살아있는 언어)
  - 똑같은 문장도 여러 형태로 표현 가능, 문장의 뜻이 예매할 수 있음.
  - 세월이 흐르면서 변화함.
- 자연어 처리 기술
  - 검색 엔진, 기계 번역, 질의 응답 시스템, IME, 문장 자동 요약, 감정 분석

## 2.1.1 단어의 의미

- 단어
  - 의미의 최소 단위
  - 컴퓨터에게 단어의 의미를 이해시키는 기술이 중요함
  
- 단어의 의미를 이해시키는 세가지 방법
  - 시소러스를 활용한 기법
  - 통계 기반 기법
  - 추론 기반 기법(word2vec)

## 2.2 시소러스

### ■ 시소러스

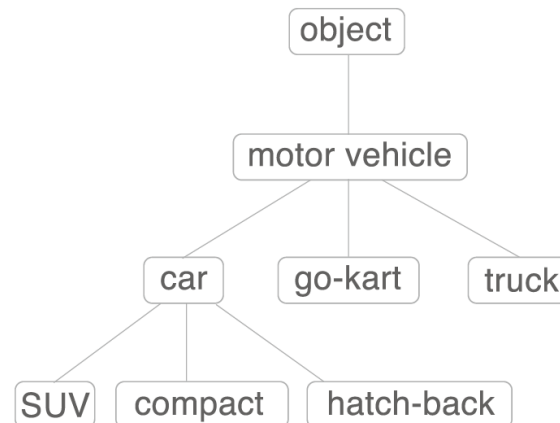
- 유의어 사전
- 단어 네트워크를 만들어서 컴퓨터에게 단어 사이의 관계를 가르침
- 동의어와 유의어가 한 그룹으로 분류됨

그림 2-1 동의어의 예: “car”, “auto”, “automobile” 등은 “자동차”를 뜻하는 동의어다.



- 자연어 처리에서는 단어 사이의 관계를 정의  
(상위와 하위, 전체와 부분)

그림 2-2 단어들을 의미의 상·하위 관계에 기초해 그래프로 표현한다(문헌 [14]를 참고하여 그림).



## 2.2.1 WordNet

### ■ WordNet

- 자연어 처리 분야에서 가장 유명한 시소러스
- 프린스턴 대학교에서 1985년 부터 구축
- 다양한 자연어 처리 어플리케이션에서 활용
- 유의어를 얻거나 단어 네트워크를 이용할 수 있음
- 단어 사이의 유사도를 구할 수 있음

## 2.2.2 시소러스의 문제점

- 시대 변화에 대응하기 어렵다
  - 신조어가 생겨나고, 옛말은 잊혀짐
  - 시대에 따라 언어의 의미가 변함
  - 단어의 변화에 대응하려면 시소러스를 수작업으로 끊임없이 갱신 필요
- 사람을 쓰는 비용이 크다
  - 시소러스를 만드는 데는 엄청난 인적 비용이 발생함
  - 영어 단어수는 1,000만개 넘는데 이 단어 사이의 관계를 정의 해야함
- 단어의 미묘한 차이를 표현 할 수 없다
  - 비슷한 단어들이라도 미묘한 차이가 있음
  - 미묘한 차이를 수작업으로 표현하는 것은 어려운 일임
- 시소러스의 문제점 해결 방법
  - 통계 기반 기법, 신경망을 사용한 추론 기반 기법
  - 대량의 텍스트 데이터로 부터 단어의 의미를 자동으로 추출

## 2.3 통계 기반 기법

### ■ 말뭉치 (Corpus)

- 대량의 텍스트 데이터
- 그 안에 담긴 문장들은 사람이 쓴 글임
- 자연어 처리 연구를 염두에 두고 수집된 텍스트 데이터
- 자연어에 대한 사람의 지식이 충분히 담겨 있음
- 문장을 쓰는 방법, 단어를 선택하는 방법, 단어의 의미 등

### ■ 통계 기반 기법의 목표

- 사람의 지식으로 가득한 말뭉치에서 자동으로, 효율적으로 그 핵심을 추출하는 것임

## 2.3.1 파이썬으로 말뭉치 전처리하기

- 자연어 처리에 사용되는 다양한 말뭉치
  - 위키백과, 구글 뉴스, 셰익스피어 같은 대문호의 작품 등등
- 전처리 수행
  - 말뭉치 예시

```
>>> text = 'You say goodbye and I say hello.'
```

- 위의 텍스트를 단어 단위로 분할

```
>>> text = text.lower()
>>> text = text.replace('.', ' .')
>>> text
'you say goodbye and i say hello .'
>>> words = text.split(' ')
>>> words
['you', 'say', 'goodbye', 'and', 'i', 'say', 'hello', '.']
```

- 원래의 문장을 단어 목록 형태로 이용할 수 있게 됨
- 단어를 텍스트 그대로 조작하기란 여러 면에서 불편함

## 2.3.1 파이썬으로 말뭉치 전처리하기(1)

- 단어에 ID 부여하고, ID의 리스트로 이용
  - 딕셔너리를 이용하여 단어 ID와 단어를 짝짓는 대응표 작성

```
>>> word_to_id = {}  
>>> id_to_word = {}  
>>>  
>>> for word in words:  
...     if word not in word_to_id:  
...         new_id = len(word_to_id)  
...         word_to_id[word] = new_id  
...         id_to_word[new_id] = word
```

- 단어 ID와 단어의 대응표 확인

```
>>> id_to_word  
{0: 'you', 1: 'say', 2: 'goodbye', 3: 'and', 4: 'i', 5: 'hello', 6: '.'}  
>>> word_to_id  
{'you': 0, 'say': 1, 'goodbye': 2, 'and': 3, 'i': 4, 'hello': 5, '.': 6}
```

- 단어를 가지고 단어 ID 검색, 단어 ID를 가지고 단어를 검색

```
>>> id_to_word[1]  
'say'  
>>> word_to_id['hello']  
5
```

## 2.3.1 파이썬으로 말뭉치 전처리하기(2)

- 단어 목록을 단어 ID 목록으로 변경
  - 단어 목록에서 단어 ID 목록으로 변환한 다음 넘파이 배열로 변환

```
>>> text = 'You say goodbye and I say hello.'
```

```
>>> import numpy as np
>>> corpus = [word_to_id[w] for w in words]
>>> corpus = np.array(corpus)
>>> corpus
array([0, 1, 2, 3, 4, 1, 5, 6])
```

- 내포 - 리스트나 딕셔너리 등의 반복문 처리를 간단하게 쓰기 위한 기법
- 말뭉치를 이용하기 위한 전처리 완료

## 2.3.1 파이썬으로 말뭉치 전처리하기(3)

- 말뭉치 전처리를 위한 preprocess() 함수 구현
  - 단어 목록에서 단어 ID 목록으로 변환한 다음 넘파이 배열로 변환

```
def preprocess(text):  
    text = text.lower()  
    text = text.replace('.', ' .')  
    words = text.split(' ')  
  
    word_to_id = {}  
    id_to_word = {}  
    for word in words:  
        if word not in word_to_id:  
            new_id = len(word_to_id)  
            word_to_id[word] = new_id  
            id_to_word[new_id] = word  
  
    corpus = np.array([word_to_id[w] for w in words])  
    return corpus, word_to_id, id_to_word
```

- 내포 - 리스트나 딕셔너리 등의 반복문 처리를 간단하게 쓰기 위한 기법
- 말뭉치를 이용하기 위한 전처리 완료

## 2.3.2 단어의 분산 표현

### ■ 색(Color)의 벡터 표현

- 색은 가짓수 만큼 이름 부여 - 코발트블루, 싱크레드, 진홍
- 색의 3차원 벡터 표현 – RGB 성분을 비율로 표현
- RGB 벡터 표현이 색을 명확하게 명시, 관련성도 쉽게 판단, 정량화 쉬움
- 비색 vs  $(R, G, B) = (170, 33, 22)$

### ■ 단어의 분산 표현

- 색을 벡터로 표현하듯 단어도 벡터로 표현
- 단어의 의미를 정확하게 파악할 수 있는 벡터 표현
- 단어를 고정 길이의 밀집 벡터로 표현

## 2.3.3 분포 가설

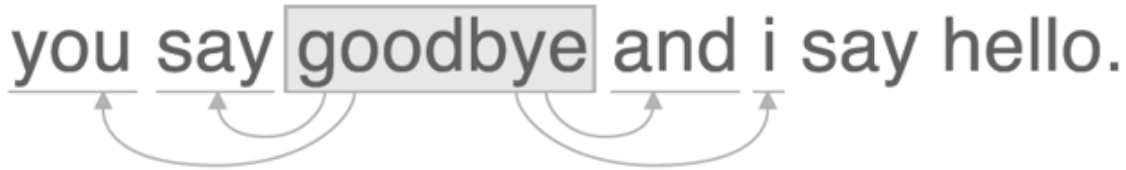
### ■ 분포 가설(Distributional Hypothesis) 이란?

- 아이디어 – 단어의 의미는 주변 단어에 의해 형성된다
- 단어 자체는 의미가 없고, 그 단어가 사용된 맥락이 의미를 형성
- "guzzle"과 "drink"는 가까운 의미의 단어
  - "I drink beer" 와 "We drink beer", "I guzzle beer"와 "We guzzle wine"

### ■ 맥락 (Context)

- 이번장에서 맥락이라 하면 (주목하는 단어) 주변에 놓인 단어를 가르킴

you say goodbye and i say hello.



- 특정 단어를 중심에 둔 그 주변 단어를 말함
- 윈도우 크기 (맥락의 크기) – 주변 단어를 몇 개나 포함할지를 나타냄

## 2.3.4 동시발생 행렬

- 단어를 벡터로 나타내는 방법
  - 주변 단어를 세어보는 방법
  - 통계 기반 기법
    - 주목한 단어 주변에 어떤 단어가 몇 번이나 등장하는지 세어 집계함
- 통계 기반 기법
  - 말뭉치와 preprocess( ) 함수를 사용해 전처리

```
import sys
sys.path.append('.')
import numpy as np
from common.util import preprocess

text = "You say googbye and I say hello."
corpus, word_to_id, id_to_word = preprocess(text)
print(corpus)
print(id_to_word)
```

```
[0 1 2 3 4 1 5 6]
```

```
{0: 'you', 1: 'say', 2: 'googbye', 3: 'and', 4: 'i', 5: 'hello', 6: '.'}
```

## 2.3.4 동시발생 행렬 (1)

### ■ 통계 기반 기법

- 각 단어의 맥락에 해당하는 단어의 빈도를 세어봄

그림 2-4 단어 "you"의 맥락을 세어본다.

**you** say goodbye and i say hello .



- "you"의 맥락은 "say"라는 단어 하나뿐임.

그림 2-5 단어 "you"의 맥락에 포함되는 단어의 빈도를 표로 정리한다.

|     | you | say | goodbye | and | i | hello | . |
|-----|-----|-----|---------|-----|---|-------|---|
| you | 0   | 1   | 0       | 0   | 0 | 0     | 0 |

- "you"의 맥락으로써 동시에 발생하는 단어의 빈도를 나타냄
- "you"라는 단어를 [0, 1, 0, 0, 0, 0, 0] 라는 벡터로 표현

## 2.3.4 동시발생 행렬 (2)

### ■ 통계 기반 기법

- ID 가 1인 "say"에 대해서도 같은 작업을 수행

그림 2-6 단어 "say"의 맥락에 포함되는 단어의 빈도를 표로 정리한다.



|     | you | say | goodbye | and | i | hello | . |
|-----|-----|-----|---------|-----|---|-------|---|
| say | 1   | 0   | 1       | 0   | 1 | 1     | 0 |

- "say"라는 단어는 벡터 [1, 0, 1, 0, 1, 1, 0] 으로 표현

## 2.3.4 동시발생 행렬 (3)

### ■ 동시발생 행렬

- 모든 단어에 대해 동시발생하는 단어를 표에 정리함

그림 2-7 모든 단어 각각의 맥락에 해당하는 단어의 빈도를 세어 표로 정리한다.

|         | you | say | goodbye | and | i | hello | . |
|---------|-----|-----|---------|-----|---|-------|---|
| you     | 0   | 1   | 0       | 0   | 0 | 0     | 0 |
| say     | 1   | 0   | 1       | 0   | 1 | 1     | 0 |
| goodbye | 0   | 1   | 0       | 1   | 0 | 0     | 0 |
| and     | 0   | 0   | 1       | 0   | 1 | 0     | 0 |
| i       | 0   | 1   | 0       | 1   | 0 | 0     | 0 |
| hello   | 0   | 1   | 0       | 0   | 0 | 0     | 1 |
| .       | 0   | 0   | 0       | 0   | 0 | 1     | 0 |

- 이 표의 각 행은 해당 단어를 표현한 벡터가 됨
- 표가 행렬의 형태로 보임

## 2.3.4 동시발생 행렬 구현

### ■ 동시발생 행렬을 파이썬으로 구현

- 모든 단어에 대해 동시발생하는 단어를 표에 정리함

```
C = np.array([
    [0, 1, 0, 0, 0, 0, 0],
    [1, 0, 1, 0, 1, 1, 0],
    [0, 1, 0, 1, 0, 0, 0],
    [0, 0, 1, 0, 1, 0, 0],
    [0, 1, 0, 1, 0, 0, 0],
    [0, 1, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 1, 0],
], dtype=np.int32)
```

- 동시발생 행렬을 활용하면 단어를 벡터로 나타낼 수 있음

```
print(C[0]) # ID가 0인 단어의 벡터 표현
```

```
[0 1 0 0 0 0 0]
```

```
print(C[word_to_id['goodbye']]) # "goodbye"의 벡터 표현
```

```
[0 1 0 1 0 0 0]
```

## 2.3.4 동시발생 행렬 구현(1)

### ■ 동시발생 행렬 함수 구현

- 말뭉치로부터 동시발생 행렬을 만들어주는 함수를 구현

```
def create_co_matrix(corpus, vocab_size, window_size=1):  
    corpus_size = len(corpus)  
    co_matrix = np.zeros((vocab_size, vocab_size), dtype=np.int32)  
  
    for idx, word_id in enumerate(corpus):  
        for i in range(1, window_size + 1):  
            left_idx = idx - i  
            right_idx = idx + i  
  
            if left_idx >= 0:  
                left_word_id = corpus[left_idx]  
                co_matrix[word_id, left_word_id] += 1  
  
            if right_idx < corpus_size:  
                right_word_id = corpus[right_idx]  
                co_matrix[word_id, right_word_id] += 1  
  
    return co_matrix
```

### ■ 소스 분석

- 0으로 채워진 2차원 배열로 초기화
- 말뭉치의 모든 단어 각각에 대해서 윈도우에 포함된 주변 단어를 세어감
- 자동으로 동시발생 행렬을 만들어 줌.

## 2.3.5 벡터 간 유사도

- 벡터 사이의 유사도를 측정하는 방법
  - 벡터의 내적, 유클리드 거리, 코사인 유사도

- 코사인 유사도

- 두 벡터  $x = (x_1, x_2, \dots, x_n)$ 과  $y = (y_1, y_2, \dots, y_n)$ 의 코사인 유사도

$$\text{similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = \frac{x_1 y_1 + \dots + x_n y_n}{\sqrt{x_1^2 + \dots + x_n^2} \sqrt{y_1^2 + \dots + y_n^2}}$$

- 분자는 벡터의 내적, 분모에는 각 벡터의 L2 노름(norm)
  - 복잡한 함수의 미분을 계산하는데 유용함

## 2.3.5 벡터 간 유사도(1)

### ■ 코사인 유사도를 파이썬 함수로 구현

```
def cos_similarity(x, y):  
    nx = x / (np.sqrt(np.sum(x ** 2))) # x의 정규화  
    ny = y / (np.sqrt(np.sum(y ** 2))) # y의 정규화  
    return np.dot(nx, ny)
```

- 벡터 x와 y를 정규화한 후 두 벡터의 내적을 구함
- 인수로 제로 벡터가 들어오면 '0으로 나누기' 오류가 발생함
- 이 문제를 해결하는 방법은 나눌 때 작은 값을 더해줌

```
def cos_similarity(x, y, eps=1e-8):  
    nx = x / (np.sqrt(np.sum(x ** 2)) + eps)  
    ny = y / (np.sqrt(np.sum(y ** 2)) + eps)  
    return np.dot(nx, ny)
```

## 2.3.5 벡터 간 유사도(2)

- 벡터의 유사도를 구하는 코드
  - "you"와 "i(=I)"의 유사도를 구하는 코드

```
import sys
sys.path.append('.')
from common.util import preprocess, create_co_matrix, cos_similarity

text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)
vocab_size = len(word_to_id)
C = create_co_matrix(corpus, vocab_size)

c0 = C[word_to_id['you']] # "you"의 단어 벡터
c1 = C[word_to_id['i']]  # "i"의 단어 벡터
print(cos_similarity(c0, c1))
```

0.7071067691154799

- "you"와 "i"의 코사인 유사도는 0.707.. 으로 나옴

## 2.3.6 유사 단어의 랭킹 표시

### ■ 유사 단어 출력하는 함수 구현

- 함수 설명 - 어떤 단어가 검색어로 주어지면, 그 검색어와 비슷한 단어를 유사도 순으로 출력하는 함수

- 1) 검색어의 단어 벡터를 꺼냄
- 2) 검색어의 단어 벡터와 다른 모든 단어 벡터와의 코사인 유사도를 각각 구함
- 3) 계산한 코사인 유사도 결과를 기준으로 값이 높은 순서대로 출력

```
def most_similar(query, word_to_id, id_to_word, word_matrix, top=5):  
    # 1) 검색어를 꺼낸다  
    if query not in word_to_id:  
        print('%s(을)를 찾을 수 없습니다.' % query)  
        return  
  
    print('\n[query] ' + query)  
    query_id = word_to_id[query]  
    query_vec = word_matrix[query_id]  
  
    # 2) 코사인 유사도 계산  
    vocab_size = len(id_to_word)  
    similarity = np.zeros(vocab_size)  
    for i in range(vocab_size):  
        similarity[i] = cos_similarity(word_matrix[i], query_vec)  
  
    # 3) 코사인 유사도를 기준으로 내림차순으로 출력  
    count = 0  
    for i in (-1 * similarity).argsort():  
        if id_to_word[i] == query:  
            continue  
        print(' %s: %s' % (id_to_word[i], similarity[i]))  
        count += 1  
        if count >= top:  
            return
```

## 2.3.6 유사 단어의 랭킹 표시 (1)

- 유사 단어 출력하는 함수 구현
  - “you” 를 검색어로 지정해 유사한 단어를 출력

```
import sys
sys.path.append('.')
from common.util import preprocess, create_co_matrix, most_similar

text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)
vocab_size = len(word_to_id)
C = create_co_matrix(corpus, vocab_size)

most_similar('you', word_to_id, id_to_word, C, top=5)
```

```
[query] you
goodbye: 0.7071067691154799
i: 0.7071067691154799
hello: 0.7071067691154799
say: 0.0
and: 0.0
```

## 2.4 통계 기반 기법 개선하기

- 동시발생 행렬의 개선점
- 상호정보량
  - 동시발생 행렬의 원소는 두 단어가 동시에 발생한 횟수를 나타냄
  - 발생 횟수 특징의 문제점
    - "the"와 "car" – 두 단어의 동시발생 횟수는 아주 많음
    - "car"와 "drive" – 두 단어의 관련이 깊지만, 등장 횟수로 보면 "the"가 "drive" 보다 관련성이 훨씬 강하다고 나옴
    - "the" 가 고빈도 단어라서 "car"와 강한 관련성을 갖는다고 평가

## 2.4.1 상호정보량

### ■ 점별 상호정보량(PMI) 척도

- PMI는 확률 변수  $x$  와  $y$ 에 대해 다음 식으로 정의됨

$$\text{PMI}(x, y) = \log_2 \frac{P(x, y)}{P(x)P(y)}$$

- PMI 값이 높을수록 관련성이 높다는 의미임

### ■ 동시발생 확률을 사용한 식

- $C$ 는 동시발생 행렬,  $C(x, y)$ 는 단어  $x$ 와  $y$ 가 동시발생하는 횟수
- $C(x)$ 와  $C(y)$ 는 각각 단어  $x$ 와  $y$ 의 등장 횟수,  $N$ 은 말뭉치에 포함된 단어수

$$\text{PMI}(x, y) = \log_2 \frac{P(x, y)}{P(x)P(y)} = \log_2 \frac{\frac{C(x, y)}{N}}{\frac{C(x)}{N} \frac{C(y)}{N}} = \log_2 \frac{C(x, y) \cdot N}{C(x)C(y)}$$

## 2.4.1 상호정보량(1)

### ■ PMI 계산

- 말뭉치 단어수  $N$ 이 10,000 이고, "the"와 "car"와 "drive"가 각각 1,000번, 20번, 10번 등장, "the"와 "car"의 동시 발생 수는 10회, "car"와 "drive"의 동시 발생수는 5회일 때 PMI 계산 결과는?

$$\text{PMI}(\text{"the"}, \text{"car"}) = \log_2 \frac{10 \cdot 10000}{1000 \cdot 20} \approx 2.32$$

$$\text{PMI}(\text{"car"}, \text{"drive"}) = \log_2 \frac{5 \cdot 10000}{20 \cdot 10} \approx 7.97$$

- 단어가 단독으로 출현하는 횟수가 고려되어, "the"가 자주 출현함으로 PMI 점수가 낮아짐.
- 문제는 동시발생 횟수가 0이면  $\log_2 0 = -\infty$  가 됨.

### ■ 양의 상호정보량(PPMI)

- PMI가 음수일 때는 0으로 취급함

$$\text{PPMI}(x, y) = \max(0, \text{PMI}(x, y))$$

## 2.4.1 상호정보량(2)

### ■ PPMI 행렬로 변환함수 구현

#### ■ 동시발생 행렬을 PPMI 행렬로 변환하는 함수 구현

```
def ppmi(C, verbose=False, eps = 1e-8):
    M = np.zeros_like(C, dtype=np.float32)
    N = np.sum(C)
    S = np.sum(C, axis=0)
    total = C.shape[0] * C.shape[1]
    cnt = 0

    for i in range(C.shape[0]):
        for j in range(C.shape[1]):
            pmi = np.log2(C[i, j] * N / (S[j]*S[i]) + eps)
            M[i, j] = max(0, pmi)

            if verbose:
                cnt += 1
                if cnt % (total//100 + 1) == 0:
                    print('%0.1f%% 완료' % (100*cnt/total))

    return M
```

```
# coding: utf-8
import sys
sys.path.append('..')
import numpy as np
from common.util import preprocess, create_co_matrix, cos_similarity, ppmi

text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)
vocab_size = len(word_to_id)
C = create_co_matrix(corpus, vocab_size)
W = ppmi(C)

np.set_printoptions(precision=3) # 유효 자릿수를 세 자리로 표시
print('동시발생 행렬')
print(C)
print('-'*50)
print('PPMI')
print(W)
```

동시발생 행렬

```
[[0 1 0 0 0 0 0]
 [1 0 1 0 1 1 0]
 [0 1 0 1 0 0 0]
 [0 0 1 0 1 0 0]
 [0 1 0 1 0 0 0]
 [0 1 0 0 0 0 1]
 [0 0 0 0 0 1 0]]
```

PPMI

```
[[0.      1.807 0.      0.      0.      0.      0.      ]
 [1.807 0.      0.807 0.      0.807 0.807 0.      ]
 [0.      0.807 0.      1.807 0.      0.      0.      ]
 [0.      0.      1.807 0.      1.807 0.      0.      ]
 [0.      0.807 0.      1.807 0.      0.      0.      ]
 [0.      0.807 0.      0.      0.      0.      2.807]
 [0.      0.      0.      0.      0.      2.807 0.      ]]
```

## 2.4.1 상호정보량(3)

### ■ PPMI 행렬의 문제점

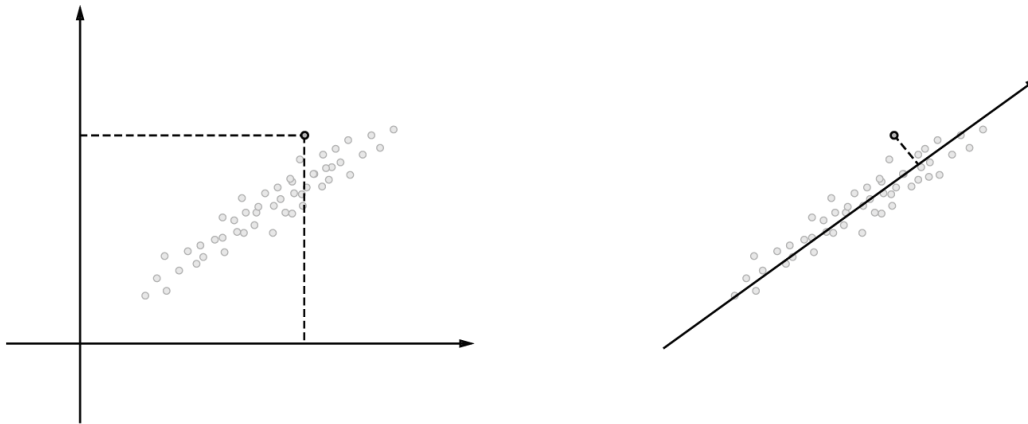
- PPMI 행렬의 각 원소는 0 이상의 실수
- 말뭉치의 어휘 수가 증가함에 따라 각 단어 벡터의 차원 수도 증가함
  - 예를 들어 말뭉치의 어휘 수가 10만개라면 그 벡터의 차원 수도 10만이 됨
- 행렬의 원소 대부분이 0임
- 각 원소의 중요도가 낮다는 뜻
- 이런 벡터는 노이즈에 약하고 견고하지 못하다는 약점이 있음
- 이런 문제를 대체하는 기법이 벡터의 차원 감소임

## 2.4.2 차원 감소

### ■ 차원 감소

- 벡터의 차원을 줄이는 방법
- 중요한 정보는 최대한 유지하면서 줄이는 게 핵심임

그림 2-8 그림으로 이해하는 차원 감소: 2차원 데이터를 1차원으로 표현하기 위해 중요한 축(데이터를 넓게 분포시키는 축)을 찾는다.



- 새로운 축을 찾을 때는 데이터가 넓게 분포되도록 고려해야 함
- 각 데이터점의 값은 새로운 축으로 사영된 값으로 변함
- 가장 적합한 축을 찾아내어 1차원 값만으로도 데이터의 본질적인 차이를 구별할 수 있어야 함

## 2.4.2 차원 감소(1)

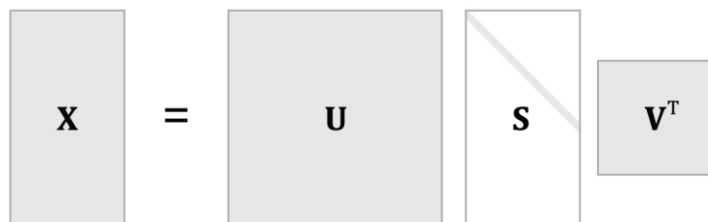
### ■ 특잇값분해 (Singular Value Decomposition)

- SVD는 임의의 행렬을 세 행렬의 곱으로 분해

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$$

- U와 V는 직교행렬이고, 그 열벡터는 서로 직교함. S는 대각 행렬

그림 2-9 SVD에 의한 행렬의 변환(행렬의 '흰 부분'은 원소가 0임을 뜻함)



- U 행렬을 단어 공간으로 취급, S는 대각 행렬로 대각성분은 특잇값이 큰 순서로 나열됨 -> 특잇값이 작은 원소를 깎아내는 방법

그림 2-10 SVD에 의한 차원 감소



## 2.4.3 SVD에 의한 차원 감소

### ■ SVD 구현

- SVD는 넘파이의 linalg 모듈에서 제공
- 동시 발생 행렬을 만들어 PPMI 행렬로 변환한 다음 SVD를 적용

```
import sys
sys.path.append('.')
import numpy as np
import matplotlib.pyplot as plt
from common.util import preprocess, create_co_matrix, ppmi
```

```
text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)
vocab_size = len(id_to_word)
C = create_co_matrix(corpus, vocab_size, window_size=1)
W = ppmi(C)
```

```
# SVD
U, S, V = np.linalg.svd(W)
```

```
print(C[0]) #동시발생 행렬
```

```
[0 1 0 0 0 0]
```

```
print(W[0]) # PPMI 행렬
```

```
[0.    1.807 0.    0.    0.    0.    ]
```

```
print(U[0]) #SVD
```

```
[-3.409e-01 -1.110e-16 -3.886e-16 -1.205e-01  0.000e+00  9.323e-01  
 2.664e-16]
```

- 희소벡터인 W[0]가 SVD에 의해 밀집벡터 U[0]로 변함

## 2.4.3 SVD에 의한 차원 감소 (1)

### ■ 그래프 그리기

- 단어를 2차원 벡터로 표현한 후 그래프 그리기
- 2차원 벡터로 줄임

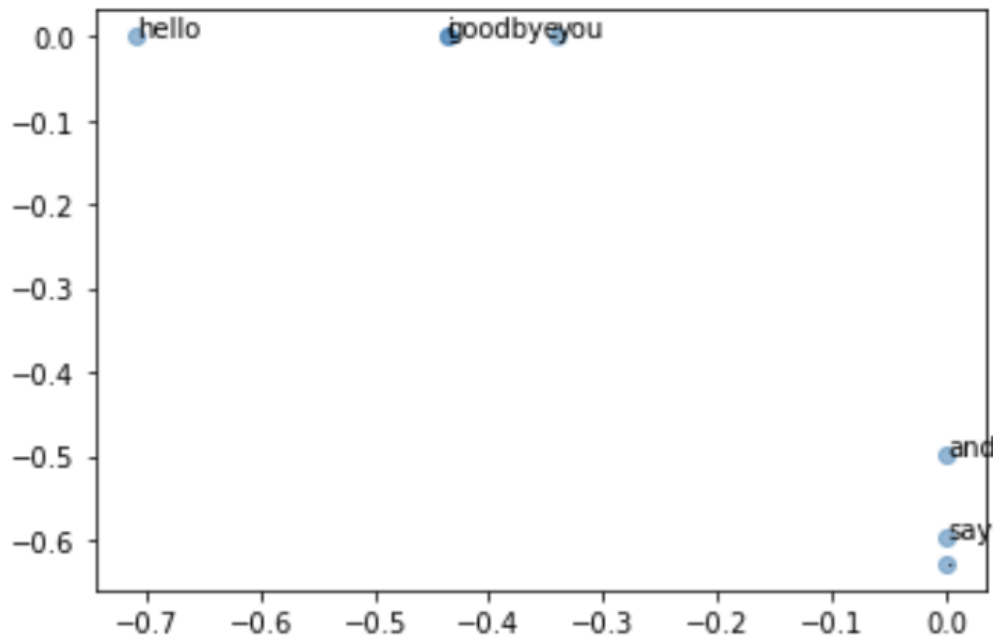
```
print(U[0, :2])
```

```
[-3.409e-01 -1.110e-16]
```

```
for word, word_id in word_to_id.items():
    plt.annotate(word, (U[word_id, 0], U[word_id, 1]))
plt.scatter(U[:,0], U[:,1], alpha=0.5)
plt.show()
```

### ■ 그래프 그리기

- "goodbye"와 "hello"
- "you"와 "I" 가 가까이 있음



## 2.4.4 PBT 데이터 셋

### ■ PBT 말뭉치

- 너무 크지 않고 적당한 크기의 말뭉치
- word2vec 발명자인 토마스 미콜로프의 웹 페이지에서 받을 수 있음
- 텍스트 파일로 제공되며 원래의 PTB 문장에 몇가지 전처리를 해둠

그림 2-12 PTB 말뭉치(텍스트 파일)의 예

```
1 consumers may want to move their telephones a little closer to the tv set
2 <unk> <unk> watching abc 's monday night football can now vote during <unk> for the greatest play in N years from
- among four or five <unk> <unk>
3 two weeks ago viewers of several nbc <unk> consumer segments started calling a N number for advice on various
- <unk> issues
4 and the new syndicated reality show hard copy records viewers ' opinions for possible airing on the next day 's show
5 interactive telephone technology has taken a new leap in <unk> and television programmers are racing to exploit the
- possibilities
6 eventually viewers may grow <unk> with the technology and <unk> the cost
```

- 한 문장이 하나의 줄로 저장
- 각 문장의 끝에 <eos> 라는 특수문자를 삽입함

## 2.4.4 PBT 데이터 셋 (1)

### ■ 코드 구현

- corpus 에는 단어 ID 목록이 저장됨.

```
import sys
sys.path.append('.')
from dataset import ptb

corpus, word_to_id, id_to_word = ptb.load_data('train')

print('말뭉치 크기:', len(corpus))
print('corpus[:30]:', corpus[:30])
print()
print('id_to_word[0]:', id_to_word[0])
print('id_to_word[1]:', id_to_word[1])
print('id_to_word[2]:', id_to_word[2])
print()
print("word_to_id['car']:", word_to_id['car'])
print("word_to_id['happy']:", word_to_id['happy'])
print("word_to_id['lexus']:", word_to_id['lexus'])
```

Downloading ptb.train.txt ...

Done

말뭉치 크기: 929589

corpus[:30]: [ 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23  
24 25 26 27 28 29]

id\_to\_word[0]: aer

id\_to\_word[1]: banknote

id\_to\_word[2]: berlitz

word\_to\_id['car']: 3856

word\_to\_id['happy']: 4428

word\_to\_id['lexus']: 7426

## 2.4.5 PTB 데이터셋 평가

- PTB 데이터셋에 통계 기반 기법을 적용
  - 큰 행렬에 SVD를 적용해야 하므로 고속의 SVD를 이용
  - 고속의 SVD를 적용하려면 sklearn 모듈을 설치
    - sklearn 의 randomized\_svd() 메서드 이용
  - 코드 - count\_method\_big.py
- 실행 결과 설명
  - "you", "year", "car", "toyota" 를 검색
  - 검색어의 단어의 의미 혹은 문법적인 관점에서 비슷한 단어들이 가까운 벡터로 나타남
  - "단어의 의미"를 벡터로 인코딩 함
  - 말뭉치를 PPMI 행렬로 변환하고 다시 SVD를 이용해 차원을 감소시켜 좋은 단어 벡터를 얻어냄

## 2.5 정리

- 이 장의 주제
  - 컴퓨터에게 “단어의 의미” 이해시키기
- 시소러스 기법
  - 단어들의 관련성을 사람이 수작업으로 하나씩 정의
- 통계 기반 기법
  - 말뭉치로부터 단어의 의미를 자동으로 추출하고 그 의미를 벡터로 표현
  - 단어의 동시발생 행렬을 만들고, PPMI 행렬로 변환
  - SVD 를 이용해 차원을 감소시켜 각 단어의 분산 표현을 만들어냄
- 분산 표현
  - 분산 표현에 따르면 비슷한 단어들이 벡터 공간에 서로 가까이 모여 있음



# Thank You

